Classes, objets, propriétés et méthodes en Swift

420-249-AH - Programmation mobile iOS

1. Programmation orientée objet en Swift

Les concepts de programmation orientée objet sont déjà connus :

- Classe
- Objet / Instance
- Propriété
- Méthode
- Encapsulation
- Héritage
- Polymorphisme
- Interface / Protocole

L'objectif ici n'est donc pas de revoir toute la théorie de la POO. Nous allons surtout voir :

> Comment ces concepts s'écrivent et se manipulent en Swift.

Dans une application mobile, les objets peuvent représenter par exemple :

- Utilisateur
- Produit
- Message
- Notification
- Appareil
- Commande
- Position
- Compte

2. Les quatre principes fondamentaux de la POO

La programmation orientée objet repose généralement sur quatre grands principes :

```
1. Encapsulation
2. Abstraction
3. Héritage
4. Polymorphisme
```

1. Encapsulation

Regrouper les données et les comportements dans un même objet et contrôler l'accès à certaines données.

```
class Compte {
    private var solde: Double = 0

    func deposer(montant: Double) {
        if montant > 0 {
            solde += montant
        }
    }
}
```

L'objet contrôle lui-même la façon dont son état peut être modifié.

2. Abstraction

Exposer ce qui est nécessaire et cacher les détails internes de l'implémentation.

Par exemple :

```
appareil.connecter()
```

Le code qui utilise l'objet n'a pas nécessairement besoin de connaître toutes les étapes internes nécessaires pour établir la connexion.

```
Utilisation simple
        ↓
    connecter()
        ↓
Détails internes cachés
```

Les protocoles (interfaces) jouent notamment un rôle important dans l'abstraction en Swift. En Swift, il n'existe pas de mot-clé interface comme dans certains autres langages.

3. Héritage

Une classe peut hériter des propriétés et comportements d'une autre classe.

```
class Appareil {
    var nom: String = ""
}
```

Puis :

```
class Telephone: Appareil {
    var numeroTelephone: String = ""
}
```

On obtient :

```
Appareil
   ↑
Telephone
```

Nous verrons plus tard :

```
super
override
final
```

4. Polymorphisme

Plusieurs objets peuvent être manipulés à travers un même type tout en ayant des comportements différents.

Exemple conceptuel :

```
Appareil
   │
   ├── Telephone
   ├── Tablette
   └── Montre
```

Chaque type peut fournir sa propre version d'un comportement.

```
func afficherInformations()
```

Le même appel peut donc produire un comportement différent selon l'objet réel.

3. Déclarer une classe

En Swift, une classe est déclarée avec le mot-clé :

```
class
```

Exemple :

```
class Utilisateur {
}
```

Ici, Utilisateur est le nom de la classe. Pour l'instant, la classe ne contient aucune donnée ni aucun comportement.

4. Ajouter des propriétés

Une propriété représente une donnée appartenant à un objet.

Exemple :

```
class Utilisateur {
    var nom: String
    var age: Int
}
```

La classe contient maintenant deux propriétés : nom, age. Chaque instance de Utilisateur possédera ses propres valeurs.

5. Initialiser les propriétés

En Swift, une propriété stockée doit généralement avoir une valeur avant que l'objet puisse être utilisé.

Une possibilité consiste à donner directement une valeur par défaut :

```
class Utilisateur {
    var nom: String = "Inconnu"
    var age: Int = 0
}
```

Il est alors possible de créer un objet :

```
let utilisateur = Utilisateur()
```

6. Créer un objet

À partir de :

```
class Utilisateur {
    var nom: String = "Inconnu"
    var age: Int = 0
}
```

on peut créer une instance :

```
let utilisateur = Utilisateur()
```

On peut représenter cela ainsi :

```
Classe Utilisateur
        ↓
     instance
        ↓
    utilisateur
```

7. Accéder aux propriétés

Swift utilise la notation pointée :

```
objet.propriete
```

Exemple :

```
let utilisateur = Utilisateur()
print(utilisateur.nom)
print(utilisateur.age)
```

Résultat :

```
Console
Inconnu
0
```

8. Modifier une propriété

Si la propriété est déclarée avec var, elle peut être modifiée.

```
let utilisateur = Utilisateur()
utilisateur.nom = "Alex"
utilisateur.age = 25
```

Puis :

```
print(utilisateur.nom)
```

Résultat :

```
Console
Alex
```

9. let sur l'objet et var sur les propriétés

Considérons :

```
let utilisateur = Utilisateur()
```

On pourrait penser que rien ne peut être modifié parce que l'objet est déclaré avec let. Pourtant :

```
utilisateur.nom = "Alex"
```

est permis si nom est une propriété var.

Pourquoi ? Parce qu'avec une classe, let empêche surtout de remplacer la référence par une autre instance.

10. Propriété constante avec let

Exemple :

```
let utilisateur = Utilisateur()
utilisateur.nom = "Alex"
utilisateur.age = 25
```

est valide.

Mais ceci ne l'est pas :

```
utilisateur = Utilisateur()
```

La constante utilisateur ne peut pas référencer un nouvel objet. Cette distinction deviendra encore plus importante lorsque nous comparerons class et struct.

Une propriété peut également être constante :

```
class Application {
    let identifiant: String = "com.exemple.mobile"
    var version: String = "1.0"
```

```
}
```

Après création :

```
let app = Application()
```

Ceci est permis :

```
app.version = "1.1"
```

Mais pas :

```
app.identifiant = "autre.identifiant"
```

11. Ajouter une méthode

Une méthode est une fonction appartenant à une classe. Reprenons une classe avec des propriétés initialisées par défaut :

```
class Appareil {
    var modele: String = "iPhone"
    var batterie: Int = 100

    func afficherEtat() {
        print("Appareil : \(modele)")
        print("Batterie : \(batterie)%")
    }
}
```

12. Appeler une méthode

```
let appareil = Appareil()
appareil.afficherEtat()
```

Résultat :

```
Console
Appareil : iPhone
Batterie : 100%
```

La syntaxe ressemble à celle utilisée pour accéder aux propriétés :

```
objet.methode()
```

13. Une méthode peut modifier l'état de l'objet

```
class Appareil {
    var batterie: Int = 100

    func utiliser() {
        batterie -= 10
    }
}
```

Utilisation :

```
let appareil = Appareil()
appareil.utiliser()
appareil.utiliser()
print(appareil.batterie)
```

Résultat :

```
Console
80
```

L'objet conserve donc son état entre les appels.

Autre exemple, avec une méthode d'affichage en plus :

```
class Batterie {
    var niveau: Int = 100

    func consommer() {
        niveau -= 10
    }

    func afficherNiveau() {
        print("Batterie : \(niveau)%")
    }
}
```

Utilisation :

```
let batterie = Batterie()
batterie.afficherNiveau()
batterie.consommer()
batterie.consommer()
batterie.afficherNiveau()
```

Résultat :

```
Console
Batterie : 100%
Batterie : 80%
```

14. Ajouter des paramètres à une méthode

Une méthode peut recevoir des paramètres comme une fonction normale.

```
class Batterie {
    var niveau: Int = 100

    func consommer(quantite: Int) {
        niveau -= quantite
    }
}
```

Utilisation :

```
let batterie = Batterie()
batterie.consommer(quantite: 15)
print(batterie.niveau)
```

Résultat :

```
Console
85
```

15. Une méthode peut retourner une valeur

```
class Batterie {
    var niveau: Int = 100

    func estFaible() -> Bool {
        return niveau <= 20
    }
}
```

Utilisation :

```
let batterie = Batterie()
if batterie.estFaible() {
    print("Batterie faible")
}
```

16. Exemple — Notification

```
class Notification {
    var titre: String = ""
    var message: String = ""
    var estLue: Bool = false

    func marquerCommeLue() {
```

```
        estLue = true
    }

    func afficher() {
        print("Titre : \(titre)")
        print("Message : \(message)")
        print("Lue : \(estLue)")
    }
}
```

Utilisation :

```
let notification = Notification()
notification.titre = "Nouveau message"
notification.message = "Vous avez reçu un message."
notification.afficher()
notification.marquerCommeLue()
notification.afficher()
```

L'objet change d'état :

```
estLue = false
      ↓
marquerCommeLue()
      ↓
estLue = true
```

17. Le mot-clé self

Swift permet d'utiliser self pour désigner l'instance courante.

Exemple :

```
class Utilisateur {
    var nom: String = ""

    func afficherNom() {
        print(self.nom)
    }
}
```

Ici, self.nom signifie : la propriété nom de cet objet.

18. self n'est pas toujours obligatoire

```
class Utilisateur {
    var nom: String = ""

    func afficherNom() {
        print(nom)
```

```
    }
}
```

Swift comprend que nom désigne la propriété de l'objet. Donc print(nom) et print(self.nom) peuvent être équivalents dans ce cas.

19. Quand self devient utile ?

self devient particulièrement utile lorsqu'un paramètre possède le même nom qu'une propriété.

```
class Utilisateur {
    var nom: String = ""

    func changerNom(nom: String) {
        self.nom = nom
    }
}
```

Ici, self.nom désigne la propriété. Et nom désigne le paramètre.

20. Comprendre self.nom = nom

```
func changerNom(nom: String) {
    self.nom = nom
}
```

on peut lire :

```
self.nom
   ↓
propriété de l'objet

nom
   ↓
paramètre reçu
```

Donc : propriété = valeur reçue.

21. Initialisation avec init

Les valeurs par défaut sont pratiques, mais elles ne conviennent pas toujours.

Supposons que chaque utilisateur doive avoir un nom différent dès sa création. Swift utilise :

```
init
```

Exemple :

```
class Utilisateur {
    var nom: String
    var age: Int
```

```
    init(nom: String, age: Int) {
        self.nom = nom
        self.age = age
    }
}
```

22. Créer un objet avec un initialisateur

```
let utilisateur = Utilisateur(
    nom: "Alex",
    age: 25
)
```

Puis :

```
print(utilisateur.nom)
print(utilisateur.age)
```

Résultat :

```
Console
Alex
25
```

23. Pourquoi utiliser init ?

Sans initialisateur, on pourrait écrire :

```
let utilisateur = Utilisateur()
utilisateur.nom = "Alex"
utilisateur.age = 25
```

Mais pendant un moment, l'objet pourrait contenir des valeurs invalides ou incomplètes.

Avec :

```
init(nom: String, age: Int)
```

on impose que les informations nécessaires soient fournies dès la création.

24. Exemple — Message

```
class Message {
    var expediteur: String
    var contenu: String
    var estLu: Bool

    init(expediteur: String, contenu: String) {
```

```
        self.expediteur = expediteur
        self.contenu = contenu
        self.estLu = false
    }
}
```

Création :

```
let message = Message(
    expediteur: "Marie",
    contenu: "Bonjour !"
)
```

Ici, expediteur et contenu doivent être fournis. Mais estLu commence automatiquement à false.

25. Initialisateur et valeur par défaut

On peut mélanger les deux approches.

```
class Notification {
    var titre: String
    var message: String
    var estLue: Bool = false

    init(titre: String, message: String) {
        self.titre = titre
        self.message = message
    }
}
```

Création :

```
let notification = Notification(
    titre: "Batterie",
    message: "Batterie faible"
)
```

La propriété estLue reçoit automatiquement false.

26. Plusieurs objets d'une même classe

Une classe peut créer plusieurs instances indépendantes.

```
let utilisateur1 = Utilisateur(
    nom: "Alex",
    age: 25
)

let utilisateur2 = Utilisateur(
    nom: "Marie",
```

```
    age: 30
)
```

Chaque objet possède ses propres données :

```
utilisateur1 → Alex, 25
utilisateur2 → Marie, 30
```

27. Modifier un objet ne modifie pas automatiquement les autres

```
utilisateur1.age = 26
```

Après cette modification :

```
utilisateur1.age → 26
utilisateur2.age → 30
```

Les deux objets sont distincts.

28. Exemple — Plusieurs appareils

```
class Appareil {
    var nom: String
    var batterie: Int

    init(nom: String, batterie: Int) {
        self.nom = nom
        self.batterie = batterie
    }

    func afficherEtat() {
        print("\(nom) : \(batterie)%")
    }
}
```

Création :

```
let iphone = Appareil(
    nom: "iPhone",
    batterie: 85
)

let ipad = Appareil(
    nom: "iPad",
    batterie: 60
)
```

Puis :

```
iphone.afficherEtat()
ipad.afficherEtat()
```

29. Objets dans un tableau

Puisque nous avons déjà vu les collections, nous pouvons stocker plusieurs objets dans un tableau.

```
let appareils: [Appareil] = [
    Appareil(nom: "iPhone", batterie: 90),
    Appareil(nom: "iPad", batterie: 70),
    Appareil(nom: "Apple Watch", batterie: 40)
]
```

Puis :

```
for appareil in appareils {
    appareil.afficherEtat()
}
```

30. Exemple — Trouver les batteries faibles

```
for appareil in appareils {
    if appareil.batterie <= 50 {
        print(
            "\(appareil.nom) : batterie faible"
        )
    }
}
```

On combine désormais : Classes + Objets + Tableaux + Boucles + Conditions.

31. Les classes sont des types

Une classe définit également un nouveau type Swift.

Par exemple :

```
class Appareil {
}
```

crée le type :

```
Appareil
```

On peut donc écrire :

```
var appareilPrincipal: Appareil
```

ou :

```
var appareils: [Appareil]
```

Tout comme on pouvait écrire :

```
var age: Int
var noms: [String]
```

32. Les propriétés peuvent être optionnelles

Une classe peut contenir des propriétés optionnelles.

Exemple :

```
class Profil {
    var nom: String
    var photo: String?

    init(nom: String) {
        self.nom = nom
        self.photo = nil
    }
}
```

Création :

```
let profil = Profil(nom: "Alex")
```

Au départ :

```
photo = nil
```

Plus tard :

```
profil.photo = "profil.jpg"
```

33. Utiliser une propriété optionnelle

```
if let photo = profil.photo {
    print("Photo : \(photo)")
} else {
    print("Aucune photo de profil")
}
```

Les notions précédentes commencent donc à s'intégrer à la POO.

34. Encapsuler un comportement dans l'objet

Considérons :

```
class Batterie {
    var niveau: Int

    init(niveau: Int) {
```

```
        self.niveau = niveau
    }
}
```

On pourrait modifier directement :

```
batterie.niveau -= 10
```

Mais il peut être préférable de placer la règle dans l'objet :

```
func consommer(_ quantite: Int) {
    niveau -= quantite
    if niveau < 0 {
        niveau = 0
    }
}
```

Ainsi :

```
batterie.consommer(10)
```

La classe contrôle elle-même son comportement.

35. Exemple complet — Appareil mobile

```
class AppareilMobile {
    var nom: String
    var batterie: Int
    var wifiActif: Bool

    init(
        nom: String,
        batterie: Int,
        wifiActif: Bool
    ) {
        self.nom = nom
        self.batterie = batterie
        self.wifiActif = wifiActif
    }

    func afficherEtat() {
        print("------ APPAREIL ------")
        print("Nom : \(nom)")
        print("Batterie : \(batterie)%")
        print("Wi-Fi : \(wifiActif)")
    }

    func utiliserBatterie(_ quantite: Int) {
        batterie -= quantite
        if batterie < 0 {
```

```
            batterie = 0
        }
    }

    func activerWifi() {
        wifiActif = true
    }

    func desactiverWifi() {
        wifiActif = false
    }

    func batterieFaible() -> Bool {
        return batterie <= 20
    }
}
```

36. Utilisation de la classe

```
let iphone = AppareilMobile(
    nom: "iPhone",
    batterie: 80,
    wifiActif: true
)

iphone.afficherEtat()
iphone.utiliserBatterie(30)

if iphone.batterieFaible() {
    print("Attention : batterie faible")
}

iphone.desactiverWifi()
iphone.afficherEtat()
```

37. Observer l'état de l'objet

Initialement :

```
Console
Nom : iPhone
Batterie : 80
Wi-Fi : true
```

Après :

```
iphone.utiliserBatterie(30)
iphone.desactiverWifi()
```

L'objet contient maintenant :

```
Console
Nom : iPhone
Batterie : 50
Wi-Fi : false
```

Une instance possède donc un état qui évolue pendant l'exécution.

38. Exemple guidé — Gestion d'un compte

Créer :

```
class Compte {
```

avec :

```
nomUtilisateur
solde
estActif
```

Initialisateur :

```
init(
    nomUtilisateur: String,
    solde: Double
)
```

Le compte commence avec :

```
estActif = true
```

Ajouter les méthodes :

```
afficherInformations()
deposer(...)
retirer(...)
desactiver()
```

Règles :

- Un dépôt doit être positif
- Un retrait ne peut pas dépasser le solde
- Un compte désactivé ne peut pas effectuer d'opération

L'objectif est de commencer à placer les règles de fonctionnement à l'intérieur de l'objet.

39. Exercice — Gestion d'appareils mobiles

Créer une classe :

```
class AppareilMobile
```

avec les propriétés :

Propriété	Type suggéré
`nom`	String
`batterie`	Int
`stockageDisponible`	Double
`estConnecte`	Bool

1. Initialisateur

Créer un initialisateur permettant de fournir : nom, batterie, stockageDisponible.

La propriété estConnecte doit commencer à false.

2. Méthode connecter()

Elle doit modifier estConnecte pour true et afficher :

```
Console
iPhone connecté.
```

3. Méthode deconnecter()

Elle doit modifier l'état de connexion à false.

4. Méthode utiliserBatterie()

Créer :

```
func utiliserBatterie(_ quantite: Int)
```

La batterie diminue de la quantité reçue. Elle ne doit jamais devenir négative.

5. Méthode installerApplication()

Créer :

```
func installerApplication(
    nom: String,
    taille: Double
)
```

Si suffisamment d'espace est disponible :

```
Console
Application installée : ...
```

et le stockage disponible diminue. Sinon :

```
Console
Espace de stockage insuffisant.
```

6. Méthode afficherEtat()

Afficher un résultat similaire :

```
Console
------ APPAREIL ------
Nom : iPhone
Batterie : 75 %
Stockage disponible : 52.5 Go
Connecté : true
```

7. Créer plusieurs objets

Créer au minimum : un iPhone, un iPad, une Apple Watch, avec des valeurs différentes.

Les stocker ensuite dans :

```
[AppareilMobile]
```

et parcourir le tableau pour afficher leur état.

40. Exercice — Appareils nécessitant une attention

À partir du tableau d'objets, afficher uniquement les appareils qui respectent au moins une des conditions suivantes :

- Batterie <= 20 %
- OU Stockage disponible < 10 Go

Exemple :

```
Console
Attention requise : Apple Watch
Batterie : 15 %
```

Cet exercice permet de combiner :

```
Classe
Objet
Propriétés
Initialisateur
Méthodes
Array
for
if
Bool
```

41. À retenir

Une classe est déclarée avec :

```
class NomClasse {
}
```

Une classe peut contenir des propriétés :

```
var nom: String
var age: Int
```

Les propriétés décrivent l'état de l'objet :

```
Utilisateur
  → nom
  → âge
  → courriel
```

Une classe peut contenir des méthodes :

```
func afficher() {
}
```

Les méthodes décrivent les comportements de l'objet.

Un objet est créé à partir d'une classe :

```
let utilisateur = Utilisateur(...)
```

On accède aux membres avec :

```
objet.propriete
objet.methode()
```

L'initialisateur :

```
init(...)
```

permet de construire un objet dans un état valide.

self désigne l'instance courante :

```
self.nom = nom
```

42. Vision globale

Nous pouvons maintenant faire le passage :

```
Données simples
      ↓
  Collections
      ↓
     Objets
```

Au lieu de représenter un appareil avec plusieurs variables indépendantes :

```
let nom = "iPhone"
var batterie = 80
var wifi = true
```

nous pouvons représenter une entité complète :

```
let appareil = AppareilMobile(...)
```

qui regroupe ses données et ses comportements.